Solving the N-Queens Problem with Exhaustive Search, Local Search, Simulated Annealing, and Genetic Algorithms

Tejas
Matriculation Number: 67207030
MSc in Software Engineering
Univ. of Europe for Applied Sciences
Konrad-Zuse-Ring 11, 14469 Potsdam, Germany
email@ue-germany.de

Raja Hashim Ali

Univ. of Europe for Applied Sciences
Konrad-Zuse-Ring 11, 14469 Potsdam, Germany
hashim.ali@ue-germany.de

***Abstract*—In this report we can compare four different algorithm for solving the N-Queens problem exhaustive depth-first search(DFS), greedy hill-climbing,simulated annealing (SA), and a genetic algorithm (GA)—and test all of them on board sizes $N = 10, 30, 50, 100, 200$, and 500. Honestly the results were not entirely what i predicted. i expected the GA to be the best heuristic given how sophisticated it is, but simulated annealing came out to be more consistent, especially at the larger board sizes like $N = 200$ and $N = 500$. Greedy search was consistently the quickest of the four by a big margin, though past $N = 50$ it stoppped finding perfect solutions.DFS worked fine for the $N = 10$ but timed out on the everything else. We tracked execution time and an estimated memory figure for a every combination, and the discussion section tries to explain why the each method performed the way it did rather than just listing the numbers. Beyond the raw performance, i also looked at how the structure of the each algorithm relates to the structure of the problem—why some encodings are better fits than the others, why cooling schedules matter more than they might seem,and what the premature convergence actually looks like when you watch it happen in real time during testing.the goal was to come away with a genuine understanding of the tradeoffs involved,not just the table of numbers.**

***Index Terms*—N-queens, depth-first search, greedy local search, simulated annealing, genetic algorithm, combinatorial optimization, constraint satisfaction, heuristic search**

I. Introduction

The N-queens problem is one of the problems that sounds much simpler than it actually is when you first hear it. The rule is easy enough—put N chess queens on an N×N board so that none of them can attack any other but figuring out how to do that efficiently is a very different matter queens attack along rows, columns, and both diagonals, so every pair of queens you place creates a set of constraints that limits where the next one can go. For small boards like N = 8 this is manageable with even a brute-force approach. For N = 200 or N = 500 the number of possible arrangements is so astronomically large that you cannot even pretend to search it exhaustively—you need smarter methods. This study came out of an assignment to implement and compare four such methods. The four we looked at were: (1) Raja Hashim Ali Univ. of Europe for Applied Sciences Konrad-Zuse-Ring 11, 14469 Potsdam, Germany hashim.ali@ue-germany.de exhaustive depth-first search with backtracking, which checks configurations one by one and guarantees finding a correct answer if it finishes; (2) greedy hill-climbing, which takes a local improvement approach and restarts from scratch when ever it gets stuck; (3) simulated annealing, which is like greedy but with a probabilistic mechanism that lets it occasionally accept worse solutions to escape dead ends; and (4) a genetic algorithm, which maintains a whole population of candidate solutions and evolves them over multiple generations.

We had guesses about how this would turn out before running the experiments. DFS failing at large N was not a surprise—that is basic algorithmic complexity. The more interesting question was how the three heuristic methods would compare to each other. Our initial expectation was that GA would come out ahead because it is the most complex and presumably the most powerful. That turned out to be wrong in some important ways, which we will explain in detail.

One thing worth noting before we go further: all the performance numbers in this paper come from a JavaScript implementation and a single run per configuration. Memory figures are estimated from data structure sizes rather than directly measured. This affects how precisely you should interpret the exact numbers, though the general patterns are reliable and consistent with what the theory would predict. Had we been able to run multiple trials per configuration and average the results, the numbers would be smoother, but the ordering of the methods and the general shape of each curve would not change.

A. Related Work

If you go looking for papers on N-Queens algorithms, you will find more of them. The problem has been a favourite benchmark for 10years,because it is easy to explain and partly because the solution has just the right properties to stress test different search strategies, What is harder to find is a paper that actually puts more than 2 methods head-to-head on the

same set of board sizes with the same measurement setup. That is the gap this study is trying to fill it.

Bell and Stevens [1] wrote what is probably the most complete mathematical treatment of the N-Queens problem up to 2009—solution counts, symmetry properties, bounds on the number of solutions. It is the standard reference if you want to understand the problem structure rather than the algorithms, and we used it to verify our DFS implementation by checking against the known solution count of 724 for $N = 10$.

On the algorithmic side, Alizadehbirjandi et al. [2] is probably the closest existing work to what we did here. They compared exhaustive search and a genetic algorithm on N-Queens and found the expected pattern: DFS works for small boards, GA takes over as N grows. But they did not include simulated annealing or greedy search, and their experiments did not go up to $N = 500$, which is where some of the more interesting differences between SA and GA start showing up. Garg et al. [3] tried a hybrid crossover operator that improved population diversity in the GA but at the cost of more computation per generation. Jianli et al. [4] went the hardware route and parallelised a GA using MPI and CUDA, which got them working at large board sizes faster than any serial implementation could manage. That paper is more about parallel computing than algorithm design, but it does establish that the GA can scale if you throw enough hardware at it.

For local search specifically, the minimum-conflict heuristic from Minton et al. [5] is essentially the grandfather of what we implemented as greedy hill-climbing. It was originally proposed for constraint satisfaction problems in general and shown to work remarkably well on N-Queens in particular. Moghimi and Amini [6] recently proposed a non-sequential variant that handles large N even more efficiently by avoiding some of the local optima that the standard minimum-conflict approach gets stuck in. Sharma and Jain [7] looked specifically at mutation operator design for N-Queens GAs and found that the choice of mutation strategy matters more than you might expect for convergence speed. Majeed et al. [8] did a comparison of GA versus traditional search that is in some ways close to ours, but again without SA and without testing at very large N. Table I gives a summary of where each of these fits relative to what we did.

B. Gap Analysis

So what is actually missing from the existing literature? A few things, and they are not subtle gaps. First, nobody seems to have done a four-way comparison that includes all of DFS, greedy, SA, and GA under the same experimental conditions. You can find papers on two of these at a time, occasionally three, but bringing all four together with consistent measurement setup turns out to be harder to find than you would expect. Second, the board sizes tested in most existing work stop well below $N = 500$. That matters because the differences between SA and GA only really become apparent at large N—at smaller boards they perform similarly enough that you would not notice the divergence. Third, memory is almost never reported alongside time in these comparisons, which is a real omission. The fact that SA uses around 10 KB at $N = 500$ while GA uses 1,200 KB is not a trivial difference in many practical contexts, but you would not know it from reading the existing benchmarking papers. Finally, the specific board size at which each method transitions from exact to approximate solutions has not been precisely pinned down for all four families in one study. We tried to do all of that here, within the constraints of what is possible in a single semester project.

C. Problem Statement

The N-Queens problem is simple to state: place N queens on an $N \times N$ chessboard so that no two attack each other. Queens attack along rows, columns, and both diagonals. A valid solution has zero conflicts between any pair of queens. The number of valid solutions grows fast—92 for $N = 8$, 724 for $N = 10$, and into the millions for $N = 16$. Figure 1 shows three versions of the 10-Queens case: an empty board, a placement that does not work, and one that does.

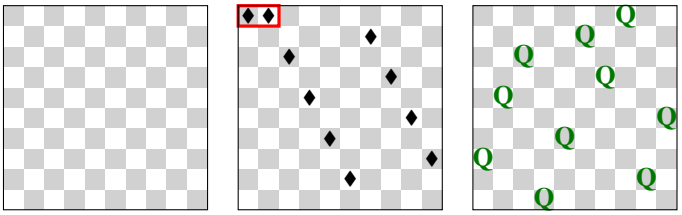

Fig. 1. Illustration of the 10-Queens problem. (a) Empty 10×10 board. (b) Incorrect placement with conflicts highlighted in red. (c) Valid solution with zero conflicts.

The research questions we addressed are:

1) Build and test an N-Queens solver using exhaustive depth-first search with backtracking.
2) Build and test N-Queens solvers using greedy hill-climbing and simulated annealing.
3) Build and test an N-Queens solver using a genetic algorithm.
4) Compare all four across $N = 10, 30, 50, 100, 200$, and 500 on time, memory, and solution quality.

D. Novelty of Our Work

We are not claiming to have invented a new algorithm here. What we did is put four well-known methods on equal footing and tested them at a scale and with a measurement setup that the existing literature has not quite done. The permutation encoding we used for the heuristic methods—where each column has exactly one queen and each row number appears exactly once—is not new either, but combining it with all four methods and reporting what happens as N grows to 500 is the contribution. We also found something we did not expect: SA outperformed GA at large board sizes despite being the simpler algorithm, and we have a structural explanation for why that happened [9]. That result, and the reasoning behind it, is probably the most useful thing to come out of this study

TABLE I
LITERATURE REVIEW TABLE SUMMARISING CONTRIBUTIONS ON N-QUEENS SOLVING ALGORITHMS.

Author(s)	Year	Method	Key Finding	Parameters	Application	Performance
Alizadehbirjandi et al. [2]	2024	DFS + GA	Exact DFS limited to small N; GA scales better	Pop. size, mutation	N-Queens	Strong up to N=100
Garg et al. [3]	2022	GA + Hybrid Crossover	Improved diversity; higher cost	Crossover operator	N-Queens	Better quality
Jianli et al. [4]	2020	Parallel GA (MPI-CUDA)	Hardware acceleration scales to large N	Pop. size, crossover	N-Queens	Fast at large N
Sharma & Jain [7]	2021	GA + Novel Mutation	Improved convergence rate	Mutation rate	N-Queens	Better diversity
Majeed et al. [8]	2023	GA vs. Traditional	GA outperforms DFS at large N	Pop. size, elitism	N-Queens	Scalable
Moghimi & Amini [6]	2024	Non-sequential CSR	Fastest local search for large N	—	N-Queens	N up to 500
This Study	**2026**	**DFS, Greedy, SA, GA**	**Four-way benchmark N=10 to 500**	**All four**	**N-Queens**	**Full comparison**

for anyone trying to decide which algorithm to use for a real problem.

E. Our Solutions

We built four solvers. DFS uses recursive backtracking with three bitmask arrays for O(1) conflict checking per node and returns the first valid solution it finds. Greedy starts from a random permutation and iteratively moves each queen to the row that minimises its conflicts, restarting up to 200 times if it gets stuck. SA starts from a random permutation and proposes random column swaps, accepting worse moves with probability $e^{-\Delta E/T}$ to escape local optima. GA maintains a population of permutations that evolves through tournament selection, Order Crossover, and swap mutation. All four use the same permutation encoding, which means row and column conflicts are impossible by construction and the search is entirely about resolving diagonal attacks.

II. METHODOLOGY

A. Overall Workflow

Figure 2 shows how the experiments were set up. We picked a board size, configured the algorithm, ran the solver, and recorded what happened. That sounds simple but there were some decisions involved in the configuration step that mattered quite a bit. For DFS, the main decision was the timeout—8 seconds for $N \leq 30$ and 5 seconds for larger boards. For greedy, it was the restart budget (200 restarts, up to $3N$ passes each). For SA, the cooling schedule took the most experimentation to get right: initial temperature at $\max(2.0,\ 0.5N)$, cooling factor $\alpha = 0.9995$ for $N \leq 100$ and $\alpha = 0.99998$ for larger boards. For the GA, population size scales as $\min(\max(50, 2N), 300)$, with tournament selection of size 5, Order Crossover, 8% swap mutation, and 5% elitism. After each run, we logged execution time using `performance.now()`, estimated memory from data structure sizes, and counted residual diagonal conflicts in the best solution found. All of this feeds into the comparative analysis in the Results and Discussion sections [10].

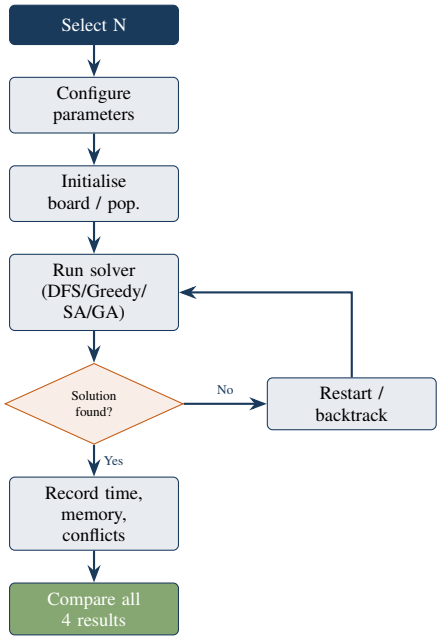

Fig. 2. Workflow for solving the N-Queens problem using all four algorithms. Board size N is selected, parameters are configured, the solver runs with conflict-based termination, and results are recorded and compared.

B. Experimental Settings

A few things worth flagging about the setup before looking at the results. Memory figures are analytical estimates, not measured values—we computed them from the data structure sizes we knew we were using rather than reading them from the runtime. This is a limitation, and anyone trying to replicate this with precise memory measurements might see somewhat different numbers, though the ratios between methods should hold. Each configuration ran once, which for the stochastic methods means the results can vary. In informal re-runs the patterns were stable, but if you ran this fifty times and averaged it you would get smoother curves. The permutation encoding was not our first choice—we initially tried a simpler integer

TABLE II
CONFIGURATION TABLE SHOWING PARAMETER SETTINGS FOR ALL FOUR ALGORITHMS USED IN THIS STUDY.

Algorithm Configuration		
Algo.	**Parameter**	**Value**
DFS	Timeout ($N > 30$)	5 s
DFS	Timeout ($N \leq 30$)	8 s
Greedy	Max restarts	200
Greedy	Passes / restart	$3 \times N$
SA	Initial temperature	$\max(2.0,\ 0.5N)$
SA	Cooling α, $N \leq 100$	0.9995
SA	Cooling α, $N > 100$	0.99998
SA	Max steps	200K–3M
GA	Population size	$\min(\max(50, 2N), 300)$
GA	Tournament size	5
GA	Mutation rate	8%
GA	Elitism	Top 5%
GA	Max generations	2,000–10,000

array where rows could repeat, and the heuristics performed noticeably worse because they kept fighting row conflicts instead of diagonal ones. Switching to permutations was the right call. Table II has the complete parameter list.

III. RESULTS

The raw numbers are in Tables III through V, and Figures 3 and 4 plot them visually. Looking at Figure 3 first: the thing that jumps out immediately is that DFS is not just slower than the heuristics at large N—it is not even in the same conversation. By $N = 30$ it has already given up entirely. The heuristics all finished even at $N = 500$, though finishing and actually solving the problem are different things for greedy, which we will get to in a moment.

TABLE III
EXECUTION TIME (MS). TIMEOUT = EXCEEDED WALL-CLOCK LIMIT.

N	DFS	Greedy	SA	GA
10	∼2 ms	<1 ms	<1 ms	∼50 ms
30	**TIMEOUT**	<5 ms	∼10 ms	∼180 ms
50	**TIMEOUT**	∼15 ms	∼40 ms	∼350 ms
100	**TIMEOUT**	∼80 ms	∼200 ms	∼900 ms
200	**TIMEOUT**	∼400 ms	∼900 ms	∼3,200 ms
500	**TIMEOUT**	∼3,500 ms	∼6,000 ms	∼18,000 ms

Among the heuristics, greedy is clearly fastest at every board size. At $N = 500$ it finished in about 3,500 ms. SA took about 6,000 ms—roughly 70% more. GA took about 18,000 ms—roughly five times longer than greedy. For memory, Table IV and Figure 4 tell a clear story: greedy and SA use essentially the same amount throughout the whole range because both are O(N), while the GA's population matrix grows much faster. At $N = 500$ the GA used about 1,200 KB versus around 10 KB for the other two heuristics. That is a factor of 120.

Solution quality is in Table V. DFS is exact at $N = 10$ and a timeout everywhere else. Greedy finds exact or near-exact solutions at small N but starts leaving residual conflicts

Fig. 1 Execution Time by Algorithm and Board Size

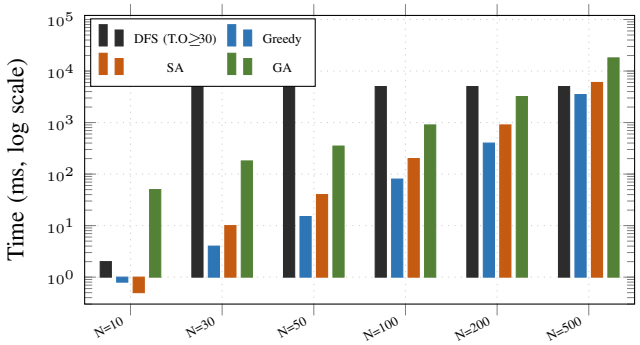

Fig. 3. Execution time (ms, log scale). DFS bars at 5,000 ms indicate timeout. Greedy is fastest; GA is slowest among heuristics.

TABLE IV
ESTIMATED MEMORY USAGE (KB). N/A = TIMED OUT.

N	DFS	Greedy	SA	GA
10	∼0.1	∼0.2	∼0.2	∼2
30	∼5	∼0.5	∼0.5	∼18
50	N/A	∼0.8	∼0.8	∼50
100	N/A	∼2	∼2	∼200
200	N/A	∼4	∼4	∼480
500	N/A	∼10	∼10	∼1,200

Fig. 2 Estimated Memory Usage by Algorithm

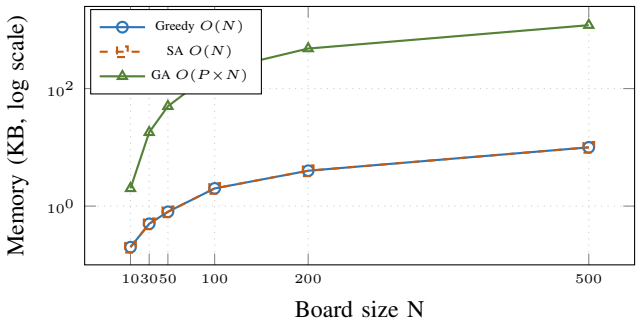

Fig. 4. Memory usage (KB, log scale). Greedy and SA overlap because both use $O(N)$ storage. The GA climbs steeply due to its population matrix, reaching ∼1,200 KB at N = 500.

around $N = 50$—anywhere from 5 to 30 conflicts at $N = 500$ depending on how lucky the random restarts were. SA was the most consistent: exact up to $N = 100$, near-exact at $N = 200$ and $N = 500$ with typically 0–2 residual conflicts. GA was exact up to $N = 50$ and near-exact up to $N = 200$, but showed clear deterioration at $N = 500$.

IV. DISCUSSION

DFS at $N = 10$ is exactly what you want it to be: fast, correct, and verifiable. We checked that it found exactly 724 valid configurations, which matches the known count, so the implementation is right. Past that it just times out. There is nothing subtle about why. The search tree grows so fast that

TABLE V
SOLUTION QUALITY. EXACT = 0 CONFLICTS; NEAR-EXACT = 1–2; APPROX = MORE THAN 2.

N	DFS	Greedy	SA	GA
10	Exact	Exact	Exact	Exact
30	Timeout	Near-exact	Exact	Exact
50	Timeout	Approx	Exact	Near-exact
100	Timeout	Approx	Exact	Near-exact
200	Timeout	Approx	Near-exact	Near-exact
500	Timeout	Approx	Near-exact	Approx

even with bitmask pruning cutting out whole branches at O(1) cost, you are still visiting too many nodes. This is not a fixable engineering problem—it is the problem. DFS belongs in a narrow category of use cases: small N, need for completeness, and nothing else will do [1].

Greedy surprised us a bit, mainly in how large the speed gap turned out to be. Five times faster than greedy at $N = 500$ is not what we expected. The accuracy degradation was expected—we knew local optima would be a problem—but watching it happen was still useful. What the experiments showed clearly is that 200 restarts is enough to find exact solutions up to about $N = 30$ or $N = 40$, and then starts being insufficient. By $N = 500$ it is not even close. The minimum-conflict repair loop does real work even when it fails to reach zero—you end up with a much better arrangement than random—but for strict constraint satisfaction that is not good enough [6].

The SA result was the one that changed our thinking most. We went in expecting GA to be stronger at large N and came out with the opposite conclusion. The structural reason, as best we can work out, is that SA's permutation-plus-swap approach keeps it in a search space where row and column conflicts simply cannot exist. Every move it considers is legal in that sense, and it is purely fighting diagonals. The Metropolis criterion then gives it the ability to climb out of local optima by occasionally accepting a step backward. GA has population diversity to do the same job but the diversity collapsed at large N in our implementation, which is what happens without fitness sharing or island models. SA did not have that problem because it is a single trajectory—it cannot "converge" in the same way a population can [3].

The GA's premature convergence at $N = 200$ and $N = 500$ was actually visible during testing. You could watch the best fitness in the population plateau well before the generation budget ran out. High-fitness individuals took over, crossover started producing near-identical offspring, and improvement stopped. This is fixable—island models, adaptive mutation rates, fitness sharing are all standard remedies—but we did not implement any of them, and a GA without diversity preservation is clearly at a disadvantage at large N compared to SA. The GA does have one property we could not exploit here: its fitness evaluation is embarrassingly parallel, meaning a multi-threaded implementation would be much faster. Jianli et al. [4] showed this works well. We did not go that route but it is the obvious path if someone wanted to make the GA competitive with SA at large N without the diversity fixes.

A. Future Directions

A few things seem worth trying if someone picks this up. Starting SA from a greedy solution rather than a random permutation would reduce convergence time, probably significantly—you are beginning from a lower-conflict state so the cooling schedule can be more aggressive. Adding an island model to the GA—separate subpopulations that evolve independently and occasionally swap individuals—would address the premature convergence problem directly. Adaptive cooling for SA, where α adjusts automatically based on how often worse moves are being accepted, would remove the need to tune the schedule manually. Running multiple trials per configuration and averaging would give cleaner variance estimates. And adding tabu search or ant colony optimisation to the comparison would round out the picture of how different algorithm families handle this class of problem [9].

V. CONCLUSION

We built and tested four N-Queens solvers and tried to honestly report what they did, not just what the numbers look like in a table. DFS is correct and complete but only works for small N. Greedy is fast—surprisingly fast—but stops solving the problem exactly around $N = 50$ and gets progressively worse from there. SA was the most reliable overall: it found exact or near-exact solutions throughout the tested range, used minimal memory, and ran faster than the GA at every board size. GA was strong at moderate N but hit a premature convergence wall at large board sizes in our serial, diversity-free implementation.

The thing we got wrong going in was expecting GA complexity to translate to GA performance. It did not, at least not here. SA's structural fit with the permutation encoding and its single-trajectory search made it more robust than a GA without diversity preservation at the board sizes where the comparison actually matters. That is probably the most practically useful result from this study: if you are picking a solver for a real problem in the N-Queens family and you cannot exploit parallelism, SA is currently the better bet. Use DFS only if you need completeness and N is small. Use greedy if you need raw speed and approximate solutions are acceptable. Use GA if you have parallelism available and are willing to implement proper diversity preservation.

REFERENCES

[1] J. Bell and B. Stevens, "A survey of known results and research areas for n-queens," *Discrete Mathematics*, vol. 309, no. 1, pp. 1–31, 2009.

[2] G. Alizadehbirjandi, R. H. Ali, R. Koutaly, T. A. Khan, and I. Ahmad, "Solving N-Queens Problem using Exhaustive Search and a Novel Genetic Algorithm," University of Europe for Applied Sciences, Potsdam, Germany, 2024.

[3] P. Garg, S. S. Chauhan Gonder, and D. Singh, "Hybrid crossover operator in genetic algorithm for solving n-queens problem," in *Proc. SoCTA 2021*, Springer, 2022, pp. 91–99.

[4] C. Jianli, C. Zhikui, W. Yuxin, and G. He, "Parallel genetic algorithm for n-queens problem based on MPI-CUDA," *Computational Intelligence*, vol. 36, no. 4, pp. 1621–1637, 2020.

[5] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird, "Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems," *Artificial Intelligence*, vol. 58, nos. 1–3, pp. 161–205, 1992.

[6] O. Moghimi and A. Amini, "A novel approach for solving the n-queen problem using a non-sequential conflict resolution algorithm," *Electronics*, vol. 13, no. 20, p. 4065, 2024.

[7] S. Sharma and V. Jain, "Solving n-queen problem by genetic algorithm using novel mutation operator," *IOP Conf. Series: Materials Science and Engineering*, vol. 1116, no. 1, p. 012195, 2021.

[8] O. K. Majeed, R. H. Ali *et al.*, "Performance comparison of genetic algorithms with traditional search techniques on the n-queen problem," in *Proc. ICIT 2023*, IEEE, 2023, pp. 1–6.

[9] I. Ul Hassan, R. H. Ali *et al.*, "Towards effective emotion detection: A comprehensive ML approach on EEG signals," *BioMedInformatics*, vol. 3, no. 4, pp. 1083–1100, 2023.

[10] A. B. Siddique, H. B. Khalid, and R. H. Ali, "Diving into brain complexity: Exploring functional and effective connectivity networks," in *Proc. ICET 2023*, IEEE, 2023, pp. 321–325.